

Rethinking Agent Design: From Top-Down Workflows to Bottom-Up Skill Evolution

Jiawei Du^{1,2} Jinlong Wu^{1,2,3} Yuzheng Chen^{1,2,3} Yucheng Hu⁴ Bing Li⁵ Joey Tianyi Zhou^{1,2}✉

¹ Centre for Frontier AI Research (CFAR), Agency for Science, Technology and Research (A*STAR), Singapore

² Institute of High Performance Computing, Agency for Science, Technology and Research (A*STAR), Singapore

³National University of Singapore, Singapore ⁴Tsinghua University, Beijing, China

⁵University of Electronic Science and Technology of China, Chengdu

Abstract

Most LLM-based agent frameworks adopt a top-down philosophy: humans decompose tasks, define workflows, and assign agents to execute each step. While effective on benchmark-style tasks, such systems rely on designer updates and overlook agents’ potential to learn from experience. Recently, Silver and Sutton [37] envision a shift into a new era, where agents could progress from a stream of experiences. In this paper, we instantiate this vision of experience-driven learning by introducing a bottom-up agent paradigm that mirrors the human learning process. Agents acquire competence through a trial-and-reasoning mechanism—exploring, reflecting on outcomes, and abstracting skills over time. Once acquired, skills can be rapidly shared and extended, enabling continual evolution rather than static replication. As more agents are deployed, their diverse experiences accelerate this collective process, making bottom-up design especially suited for open-ended environments. We evaluate this paradigm in Slay the Spire and Civilization V, where agents perceive through raw visual inputs and act via mouse outputs, the same as human players. Using a unified, game-agnostic codebase without any game-specific prompts or privileged APIs, our bottom-up agents acquire skills entirely through autonomous interaction, demonstrating the potential of the bottom-up paradigm in complex, real-world environments. Our code is available at <https://github.com/AngusDujw/Bottom-Up-Agent>.

1 Introduction

The advance in Large Language Models (LLMs) has propelled agentic AI systems toward increasingly complex task-solving capabilities [1, 14, 3, 11, 32]. Most existing agent frameworks [18, 52, 31, 30, 43] follow a top-down design philosophy: given a high-level goal, humans decompose it into subtasks, design a static or dynamic workflow, and assign agents to each node. To maintain alignment with intended goals, a memory module is often included to correct deviations. These systems [55, 36, 18, 52] are optimized for execution and correction, which makes them highly effective on well-defined, benchmark-style tasks [25, 5, 17, 33, 50], but limiting their scalability in open-ended, real-world environments.

This limitation arises from the top-down paradigm’s roots in traditional software engineering, which impose three key constraints: (i) Staticness: Agents are deployed as identical replicas of a central prototype [31, 18, 8, 10], with improvements pushed manually rather than learned adaptively. (ii) Prior Dependency: The top-down agents rely on predefined APIs [4, 43, 16, 15, 10], workflows, and task-specific prompts. In open-ended environments where such structures are unavailable, they struggle to initiate meaningful behavior. (iii) Token Utilization: A large portion of LLM tokens [46, 56] are consumed enforcing predefined workflows. These tokens could instead be used for reasoning over lived experience.

These constraints confine agent improvement to manual designer intervention, inhibiting autonomous evolution during deployment. Fortunately, the recently unlocked reasoning capabilities of LLMs enable a new generation of agents, which acquire skills not by mimicking expert trajectories, but by continuously learning from streams of grounded interactions. These agents, as envisioned by Silver and Sutton [37], mark a shift into the era of experience, in contrast to the prior era dominated by curated human data [37].

Building on this vision, we introduce a bottom-up agent paradigm that instantiates the principle of learning from experience. This paradigm leverages the reasoning capabilities of LLMs [47, 35, 14] to enable agents to acquire skills autonomously during deployment. In contrast to the execution-and-correction loop typical of top-down systems, the bottom-up paradigm prioritizes exploration and reasoning. We illustrate the distinction between top-down and bottom-up paradigms in Figure 1.

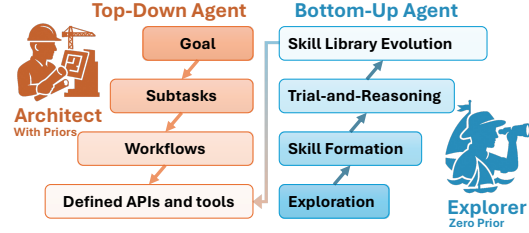


Figure 1: **Two paradigms of agent design.** Most existing agent frameworks can be categorized as **Top-down agents**, which rely on pre-engineered architectures: they begin with high-level goals, decompose them into subtasks, and execute workflows using task-specific APIs and tools. In contrast, we propose **Bottom-Up agents** to function as explorers: starting from zero prior knowledge, they gradually acquire skills through trial-and-reasoning, evolving autonomously via implicit reward inferred from environmental change.

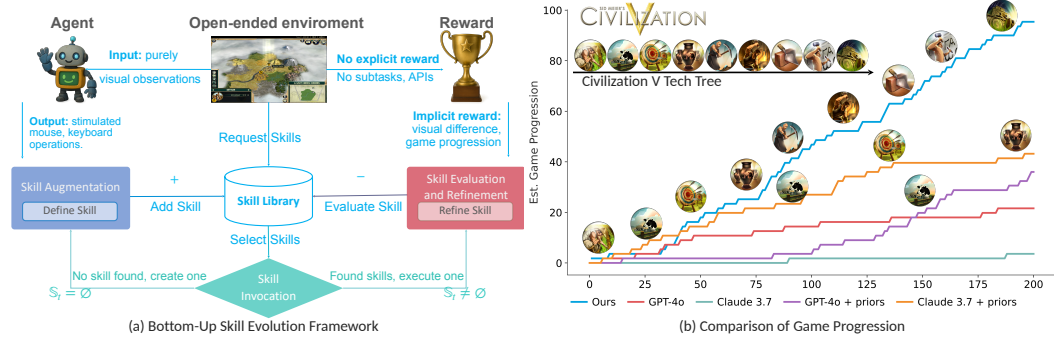


Figure 2: **Left:** The bottom-up agent operates solely on raw visual input and simulates low-level mouse and keyboard actions. Without explicit rewards, it learns and refines skills based on implicit signals like visual changes or game progression. **Right:** Game progression measured by Civilization V’s tech tree and visual changes. Our bottom-up agent (blue) outperforms all baselines, including those with task-related priors.

The bottom-up paradigm mirrors the human learning process, emphasizing experiential learning through autonomous exploration. Like humans, agents acquire competence through trial-and-reasoning interaction: they autonomously execute skills, reason about their consequences, and iteratively refine their skill libraries. Skill rewards are implicitly inferred from environmental feedback, which consists of visual changes, game progression, or numeric indicators—without requiring explicit supervision. To prevent redundant exploration and accelerate collective progress, the bottom-up paradigm also draws inspiration from the diffusion of human technologies: once a superior skill is discovered by any agent, it can be rapidly disseminated through a cloud-based knowledge-sharing mechanism. This enables all agents to immediately access and build upon the most effective strategies identified in the field, fostering a self-improving cycle of skill evolution during deployment.

Rather than replacing the top-down paradigm, the bottom-up paradigm complements it by addressing its core limitations. Agents built under this framework make full use of deployment-time LLM tokens for reasoning. This allows them to adapt and refine their skills based on real-world feedback, effectively utilizing operational tokens for reasoning. Through diverse and autonomous exploration, agents are no longer static replicas of a central prototype; instead, they function as active researchers to improve statiness. As more agents are deployed, their diverse experiences accelerate the evolution of the shared skill library. The true value of agentic systems lies not in replicable workflow design, but in this collective, experience-driven knowledge base.

As a proof of concept, we instantiate this bottom-up philosophy through skill evolution in two open-ended game environments: Slay the Spire and Civilization V. We present the bottom-up skill evolution framework in Figure 2. Unlike traditional benchmarks [25, 5, 54], these two video games

offer no explicit rewards, no predefined subgoals, and no APIs. Agents perceive the environment through visual outputs and interact via mouse operations, which is the same modality used by humans. This setup reflects the shift highlighted in The Second Half of AI [54]: from benchmarking static tasks to evaluating agents in open-ended, stream-based environments.

Experimental results show that the bottom-up agent can autonomously explore environments and successfully progress in both games. In Civilization V, the agent learns to build armies, attack City-States, and establish new cities. It completes 50 turns and unlocks 8 technologies under a zero-prior setting. In contrast, baseline agents often fail to make meaningful progress, frequently losing their starting settler or misdirecting key units such as the warrior. Similarly, in Slay the Spire, our agent clears 13 floors and achieves a 98.6% execution response rate. These results highlight the superior learning capabilities of the bottom-up paradigm and its promise for scaling to complex, real-world environments.

Our contribution is twofold: (1) we unify existing LLM-based agent frameworks under the top-down paradigm and identify three key limitations: static replication, dependence on human priors, and inefficient token usage, which constraint adaptability in open-ended settings; (2) we introduce a bottom-up paradigm based on learning from experience, offering a full formulation for agents to autonomously acquire, refine, and evolve skills through interaction.

2 Related Works

Limitations of Top-Down Workflows. The success of ChatGPT-3.5 in 2023 brought widespread attention to LLM-based agents. Yet, early agents soon encountered prompt sensitivity and hallucination issues when applied to real-world tasks. Under these constraints, the top-down paradigm was not merely a convenient design—it was an inevitable response. By decomposing complex tasks, enforcing structured workflows, and assigning agents to controlled subproblems, this approach offered a viable path to harness LLMs for reliable execution. Prominent examples include single-agent [36] systems like ReAct [55] and Plan-and-Solve [44], as well as multi-agent systems such as AutoGPT [52], MetaGPT [18], and ChatDev [31].

However, the top-down paradigm has reached a bottleneck due to three core limitations—staticness, prior dependency, and token overhead—as discussed in Figure 1. These limitations impede its ability to adapt and scale in open-ended environments. For instance, the commercialized top-down agent Manus continues to rely heavily on human-engineered workflows and toolchains to expand its functional scope, exemplifying learning from human data [37]. Moreover, once deployed, top-down agents follow highly predictable execution paths, making them vulnerable to reverse engineering. In fact, OpenManus successfully replicated Manus’s core functionalities within three hours, underscoring the brittleness of this design philosophy.

Possibilities Unlocked by Next-Gen LLMs. Recent advances in LLMs [3, 28, 14, 51] provide a foundation for rethinking agent design. First, unified understanding and generation across modalities enables agents to interpret purely visual input and express complex outputs—bridging perception and action. Second, state-of-the-art models such as DeepSeek-R1 [14], GPT-o-series [28], and Claude 3.5/3.7 [3] exhibit strong reasoning capabilities, allowing agents to reflect on consequences, revise skills, and perform introspective action evaluation. Third, executable code generation [32, 29] equips agents with the ability to synthesize and run control logic—whether for mouse/keyboard interaction in software environments or motion planning in embodied systems.

Bottom-Up Skill Evolution. These capabilities enable a shift in agent design—from imitating human data to emulating the human learning process itself. Human knowledge and skill acquisition typically unfold incrementally, shaped by lived experience rather than imposed top-down. This aligns with the “era of experience” proposed by Silver and Sutton [37], which calls for a shift from curated human trajectories to learning from unstructured streams of experience.

Some early work has explored experience-driven agents. Voyager [43] iteratively learns tool-using skills in Minecraft, but its learning process is scaffolded by task-specific prompts and privileged APIs. Open-ended RL frameworks [4, 43, 16, 15] like POET [45] and MOO [38] encourage behavioral diversity across tasks, yet they operate without the structured reasoning capabilities of modern LLMs. In contrast, we propose a bottom-up agent paradigm that leverages LLMs’ reasoning [47, 48, 35, 9]

and perception to acquire skills directly from raw experience—without predefined goals, tools (APIs), or strategy—enabling agents to evolve competence organically in open-ended environments.

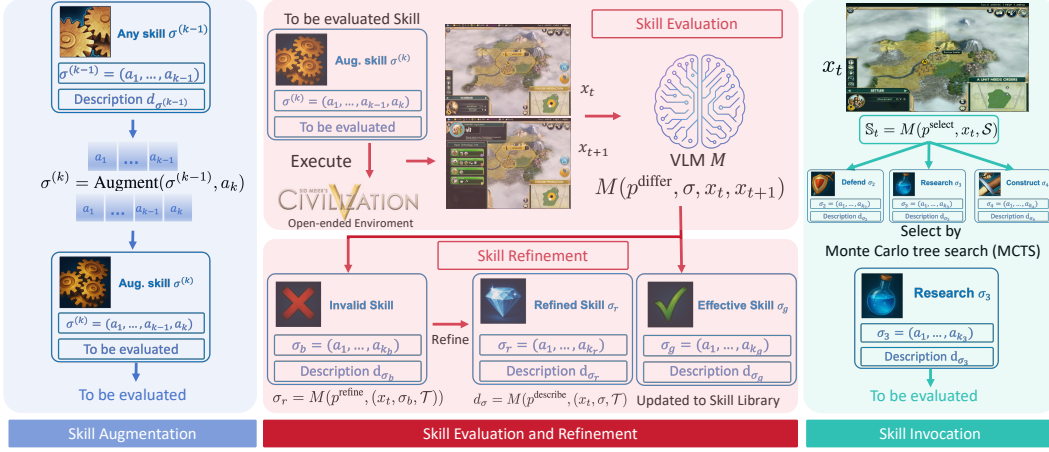


Figure 3: **Overview of Bottom-Up Skill Evolution.** The agent begins with no predefined skills and gradually builds its library $\mathcal{S}$ through interaction. **Left:** New skills are incrementally composed by extending existing routines with atomic actions. **Middle:** Skills are evaluated by a visual-language model (VLM) comparing pre- and post-execution states; ineffective ones are refined or discarded via LLM reasoning. **Right:** At each timestep, a candidate set $\mathcal{S}_t$ is selected based on the current state x_t and evaluated via Monte Carlo Tree Search (MCTS) [40] to choose the most promising skill. All components operate under a unified reasoning framework, without privileged APIs, allowing agents to acquire competence purely from experience.

3 Methodology

This section formalizes the bottom-up paradigm through the mechanism of skill evolution—the process by which agents construct, invoke, and refine skills through interaction. We begin by modeling the environment and defining the agent’s action and skill spaces. This formulation is grounded in a key principle: all complex skills can be composed from low-level human-like actions, such as mouse clicks and key presses. This compositionality forms the theoretical basis for bottom-up competence. The pseudocode of our bottom-up skill evolution can be found in [Algorithm 1](#). We also illustrate the overview of our bottom-up skill evolution in [Figure 3](#).

3.1 Problem Formulation

Notations. We model the environment as a partially observable Markov decision process (POMDP) defined by $(\mathcal{X}, \mathcal{A}, T, \mathcal{R})$, where $\mathcal{X}$ is the observation space (visual-only) and $x_t \in \mathcal{X}$ denotes the observation at time t ; $\mathcal{A}$ is the set of atomic actions (e.g., mouse clicks, drags, key presses), T denotes the unknown transition dynamics, and $\mathcal{R}$ is the implicit reward signal embedded in the environment. The agent is powered by a LLM M , which is prompt-conditioned to serve multiple roles during interaction. Executing the LLM with a prompt $p^{(f)}$ and a current context c_t yields a function-specific output: $M(p^{(f)}, c_t) \mapsto y_t^{(f)}$.

A skill is a sequence of atomic actions $\sigma = (a_1, \dots, a_k)$, where each $a_i \in \mathcal{A}$ and k controls its complexity. Each skill is paired with a semantic descriptor $d_\sigma = M(p^{\text{describe}}, (x_t, \sigma, T))$, which is a natural language summary of its intent. The skill library is denoted by $\mathcal{S} = \{\sigma_1, \dots, \sigma_n\}$ and evolves over time as new skills are discovered, refined, or composed.

Objective. We model skill evolution as a process of continual optimization over interaction sequences. The evolution of the skill library is formulated as a population-level optimization objective:

$$\max_{\mathcal{S}} \mathbb{E}_{x_t \sim \mathcal{X}, \sigma \sim \mathcal{S}} [\mathcal{R}_{\text{skill}}(\sigma, \mathcal{S}, x_t, T)] \quad \text{where } T = (x_{t+1}, \dots, x_{t+k}), \quad (1)$$

$$\text{and } \mathcal{R}_{\text{skill}}(\sigma, \mathcal{S}, x_t, T) = R_{\text{diversity}}(\sigma, \mathcal{S}) + R_{\text{efficiency}}(\sigma) + R_{\text{semantics}}(d_\sigma, T). \quad (2)$$

For any skill $\sigma = (a_1, \dots, a_k)$ invoked under state x_t , the agent receives an outcome trajectory $\mathcal{T}$, from which a behavioral signal is derived. Since no external reward $\mathcal{R}$ is available, the quality of a skill is assessed via an implicit $\mathcal{R}_{\text{skill}}$.

Each term of $\mathcal{R}_{\text{skill}}$ reflects a distinct aspect of skill quality: $R_{\text{diversity}}$ encourages semantic deviation of the skill σ from existing skills in the library $\mathcal{S}$, promoting behavioral diversity. $R_{\text{efficiency}}$ penalizes unnecessarily long or redundant action sequences, favoring concise execution. $R_{\text{semantics}}$ measures alignment between the predicted high-level effect of the skill (as described by d_σ) and the actual outcome observed in the environment.

3.2 Skill Augmentation, Invocation, and Refinement

In the bottom-up paradigm, agents do not begin with a fixed set of high-level behaviors, i.e., $\mathcal{S} = \emptyset$. They continuously augment, invoke, and refine skills through interaction with the environment, guided by reasoning from the LLM.

Skill Augmentation. In open-ended environments with no predefined APIs and priors, most atomic actions $a \in \mathcal{A}$ are task-irrelevant and meaningless. Therefore, discovering useful skills $\sigma = (a_1, \dots, a_k)$ is through a trial-and-reasoning process: the agent explores combinations of atomic actions and reflects on their outcomes to identify meaningful behaviors.

To reduce the search space, agents incrementally construct skills by increasing skill sequence length. We use k to control the length of the sequence, and it begins by evaluating single-step actions ($k = 1$), then expands to longer sequences ($k = 2, 3, \dots$), pruning unproductive branches based on behavioral feedback. Only sequences that produce recognizable effects, such as GUI transitions or task progression, are retained as meaningful candidates. The core augmentation process can thus be formalized as:

$$\sigma^{(k)} = \text{Augment}(\sigma^{(k-1)}, a_k), \quad \text{where } \sigma^{(k-1)} = (a_1, \dots, a_{k-1}), \quad a_k \in \mathcal{A} \quad (3)$$

Each new skill $\sigma^{(k)}$ is constructed by appending a random atomic action to a validated skill $\sigma^{(k-1)}$. To filter out meaningless behaviors, only sequences that trigger observable environmental changes are retained. This simple criterion ensures that the skill library preserves only potentially useful routines, enabling complex behaviors to emerge gradually from simple primitives—a core principle of bottom-up skill evolution.

Skill Invocation. Suppose multiple identical agents are concurrently exploring environments while sharing a common skill library $\mathcal{S}$. Given a new observation x_t , each agent queries the shared skill library $\mathcal{S}$ to form a candidate skill set $\mathbb{S}_t \subseteq \mathcal{S}$ using LLM reasoning:

$$\mathbb{S}_t = M(p^{\text{select}}, x_t, \mathcal{S}) \quad (4)$$

Each candidate in $\mathbb{S}_t$ represents a high-level routine potentially applicable to the current context. If no viable skill is found (i.e., $\mathbb{S}_t = \emptyset$), the agent falls back to skill augmentation in current x_t . However, due to environmental stochasticity and partial observability, agents avoid committing greedily. Instead, they evaluate $\mathbb{S}_t$ using Monte Carlo Tree Search (MCTS), simulating future rollouts to estimate the expected behavioral utility of each candidate. The most promising skill will be executed.

Skill Evaluation and Refinement. Executed skills will be evaluated by the implicit reward defined in Equation 2. Low-scoring skills are pruned from the library after repeated poor evaluations across agents. Meanwhile, when the set of viable skills becomes sparse (e.g., $\mathbb{S}_t = \emptyset$), the agent invokes the LLM to produce a semantically refined variant:

$$\sigma' = M(p^{\text{refine}}, (x_t, \sigma, \mathcal{T})) \quad (5)$$

If the new candidate σ' shows improved alignment or efficiency, it replaces the original. This two-pronged refinement, population-based pruning and LLM-guided rewriting, ensures that $\mathcal{S}$ evolves toward a more compact, reusable, and semantically consistent repertoire of skills.

4 Instantiating Bottom-Up Agents

To demonstrate the feasibility and generality of the bottom-up paradigm, we deploy a single-agent framework across two distinct open-ended games: Slay the Spire and Civilization V. These environments are intentionally chosen for their lack of explicit rewards, task-specific priors, and well-defined

APIs, where top-down agents often struggle to scale [54]. In contrast, bottom-up agents are naturally suited to such settings: their ability to autonomously explore, abstract, and refine skills allows them to operate in an environment-agnostic manner.

Perception and Control Interface. The agent perceives the environment purely through pixel-level visual observations and acts via simulated mouse inputs, closely mimicking human gameplay. At each timestep, the observation x_t is obtained as a screenshot, while actions are executed as atomic control events. Crucially, the agent has no access to privileged APIs or structured game state.

A central challenge in this setup is visual grounding, which is to interpret and act upon raw visual inputs. For instance, a click operation could theoretically target any pixel on the screen, making the action space prohibitively large. To reduce this ambiguity, we incorporate the Segment Anything Model (SAM) [21] to dynamically identify and segment UI elements and potential interaction targets. The recognized UI elements will be updated into the skill library as well. Despite this assistance, visual grounding remains one of the most brittle components in real-world deployment of bottom-up agents, often limiting their practicality and generalization.

Implicit Reward and Skill Refinement. In our setup, skill rewards are implicitly inferred from environmental feedback in two open-ended games. Specifically, we use only the semantic reward to guide skill evaluation. This is computed via the visual difference between screenshots before and after skill execution as a proxy for semantic reward, i.e., $R_{\text{semantics}} = M(p^{\text{diff}}, \sigma, x_t, x_{t+1})$. Given a new observation x_t , the agent does not generate skills from scratch; instead, it prompts the LLM to select a relevant subset $\mathcal{S}_t \subseteq \mathcal{S}$. These candidates are evaluated via a lightweight Monte Carlo Tree Search (MCTS) [40], using implicit rewards to guide selection. Skills with consistently poor performance are refined or pruned through LLM-guided revision.

The current skills operate primarily in a record-and-replay manner: they encode fixed sequences of low-level actions. While reusable within a given environment, these routines lack the functional abstraction and parameterization needed for broader generalization. One key reason is that transforming skills into callable, modular functions typically introduces environment-specific variables—such as slot bindings to UI elements or screen coordinates, which conflicts with our goal of maintaining environment-agnostic prompting and reasoning. Nevertheless, enabling such functional abstraction remains a major opportunity for improvement, as it would allow agents to generalize more flexibly and compactly across diverse scenarios.

Unified Deployment Across Environments. The same agent architecture is deployed in both Slay the Spire and Civilization V, without any environment-specific prompts, game-related rules, or prior knowledge. We expect the bottom-up agent to be truly generative to various environments. Therefore, all prompts used for skill generation, selection, and refinement are deliberately designed to be environment-agnostic, enabling the agent to apply the same reasoning logic across diverse settings without modification.

While the agent architecture and prompting remain identical, the skill library itself is currently environment-specific: skills acquired in one game are not directly transferable to another due to differences in visual semantics, action consequences, and UI layout. These turn-based games were chosen for the high latency of LLMs: they allow for tractable reasoning and skill reuse within each environment without real-time constraints, providing a suitable substrate for studying long-horizon skill evolution. Nonetheless, the underlying mechanism—visual grounding, compositional control, and LLM-based reasoning—remains general. As LLM capabilities continue to advance, we anticipate that this approach can naturally extend to more complex 3D environments and even embodied agents in the physical world.

5 Experiments

We evaluate our bottom-up agents in Slay the Spire and Civilization V, two structurally distinct open-ended games without task-specific priors or well-defined APIs. Our main results are presented in Table 1, demonstrating the evolution of skills in Table 2. We further conduct ablation studies, with findings detailed in Table 3. These evaluations are designed to address our core question: can an agent, like a human, acquire competence entirely from scratch through experience?

5.1 Experimental Setup

Environments. We evaluate LLM-based agents in two open-ended games: Slay the Spire, a roguelike deck-builder with procedural levels, and Civilization V, a turn-based strategy game involving complex planning and resource management. Both lack explicit rewards, subgoal, APIs, or task scaffolding, where top-down agents typically falter. We use the base difficulty in Slay the Spire (Ascension 0) and the “Prince” level in Civilization V, where AI opponents gain mild advantages. Their turn-based nature accommodates LLM reasoning under current latency constraints, while real-time environments are left for future work as inference efficiency improves.

Table 1: **Main results across two open-ended games.** The bottom-up agent operates under a zero-prior setting without APIs, explicit rewards, handcrafted goals, or interface bindings. As baselines struggle under these constraints, we also evaluate them with environment-specific priors (e.g., game objectives, UI control knowledge) to better highlight the potential of our bottom-up agent.

Method	Slay the Spire				Civilization V			
	Progression ↑ (Floors Cleared)	In-game ↑ Scores	Execution ↑ Responsive Rate	Token Costs ↓ (USD \$)	Progression ↑ (Turns)	Techs ↑ Researched	Execution ↑ Responsive Rate	Token Costs ↓ (USD \$)
<i>Zero Priors</i>								
GPT-4o [28]	1	5	71.4%	\$ 12.05	6	0	36.06%	\$ 7.23
Claude 3.7 [3]	1	5	26.09%	\$ 10.13	0	0	15.43%	\$ 10.08
UITARS-1.5 [32]	1	5	81.09 %	\$ 0.93	0	0	61.97 %	\$ 0.95
<i>With Priors</i>								
GPT-4o [28]	8	46	51.40%	\$ 9.80	13	1	57.24%	\$ 8.94
Claude 3.7 [3]	1	5	79.72%	\$ 12.05	17	3	92.91%	\$ 11.45
UITARS-1.5 [32]	1	5	52.59%	\$ 1.19	10	1	93.00%	\$ 1.09
<i>Zero Priors</i> Bottom-Up agent	13	81	98.56%	\$ 7.14 ¹	50	8	92.27%	\$ 6.89 ²

Zero Prior. Each agent starts with no subgoals, game-specific knowledge (not even the game name), or access to privileged APIs. Interaction begins from raw pixel inputs and proceeds via low-level atomic actions, which mirrors human gameplay. This zero-prior constraint is unique to our bottom-up setting; baselines are given basic priors (e.g., subgoals, game-specific knowledge) without which they fail to progress meaningfully. The prompts with zero prior of our bottom-up agent is presented in Figure 5. The prompts with task-specific priors of the baselines are presented in Figure 7.

LLM Backends and Deployment Protocol. We use GPT-4o as the agent backend. Each agent runs for three episodes per environment, terminating on game completion, failure, or after a fixed action limit of 1,000 steps. A run of 1,000 steps typically takes approximately 6.5 hours. All skill acquisition, invocation, and refinement steps follow the algorithmic loop defined in Algorithm 1. Importantly, both games share an identical codebase. This unified deployment highlights the generality of the bottom-up paradigm.

Baselines. Our work is the first to systematically address the problem of deploying agents in open-ended environments without APIs, subgoals, or structured observations. In such settings, most existing agent frameworks are inapplicable—they rely on predefined interfaces or task-specific prompts, and even with added priors (e.g., subgoals or game knowledge), they cannot operate effectively due to the absence of executable APIs. In practice, using these frameworks is no more effective than prompting LLMs directly. The closest comparable systems are UI agents, which interact with visual interfaces via simulated clicks and keystrokes. We therefore select GPT-4o, Claude 3.7, and the open-source UI-TARS-1.5 [32] as representative baselines for comparison.

Evaluation Metrics. We assess bottom-up agents using four metrics that capture both behavioral competence and computational efficiency across environments: (a) *Progression*: measured as floors cleared in Slay the Spire and total turns played in Civilization V, reflecting sustained engagement and survival; (b) *Strategic Development*: quantified via cumulative in-game score in Slay the Spire and number of technologies researched in Civilization V, indicating planning effectiveness; (c) *Execution Responsive Rate*: the percentage of skill invocations that lead to observable changes in game state,

¹The evaluation terminates at step 301 due to the character being defeated by the monster.

²The evaluation terminates at step 232 due to the nation being defeated by the AI.

assessing functional validity of behaviors; (d) *Token Costs*: Total LLM tokens consumed during one episode, reflecting reasoning overhead during deployment. We convert token usage into actual USD costs for better clarity.

Table 2: **Skill evolution analysis** in *Slay the Spire*. We track the growth of the skill library across multiple training rounds, where each round resumes from the previous one’s library. Metrics reflect the state of the skill library at the beginning of each round. Each round is configured with 100 action steps for clearer illustration.

Round	Skill Library Information				Slay the Spire			
	Library Size	Skills Augmented	Skills Pruned	Pruning Rate (%)	Progression ↑ (Floors Cleared)	In-game ↑ Scores	Execution ↑ Responsive Rate	Token Costs ↓ (USD \$)
Round 0	0	60	1	1.67%	6	36	93.14%	\$ 2.66
Round 1	59	16	5	31.25%	7	41	95.37%	\$ 2.28
Round 2	70	26	0	0.00%	8	53	96.73%	\$ 2.58
Round 3	96	16	1	6.25%	8	48	96.58%	\$ 2.41

5.2 Evaluation Results

We report the main results of experiments in Table 1 and visualize the comparison of game progression in Figure 2(b). We also provide demo videos as supplementary to showcase our bottom-up agents successfully mastering the two games. Compared to the baseline agents both with and without environment-specific priors, our bottom-up agent demonstrates superior performance across two distinct open-ended games. In *Slay the Spire*, where all baselines fail to progress without priors, our agent clears 13 floors and achieves a game score of 81. We outperform all baselines, including those with access to handcrafted subgoals and interface priors. Our bottom-up agent attains a 98.56% execution responsive rate, indicating that the discovered skills are not only meaningful but also highly functional. In *Civilization V*, our agent completes 50 turns and unlocks 8 technologies, showcasing effective exploration even under zero-prior constraints. While baseline performance improves marginally with added priors, they still fall short of matching our agent’s ability to sustain and adapt behaviors across timesteps.

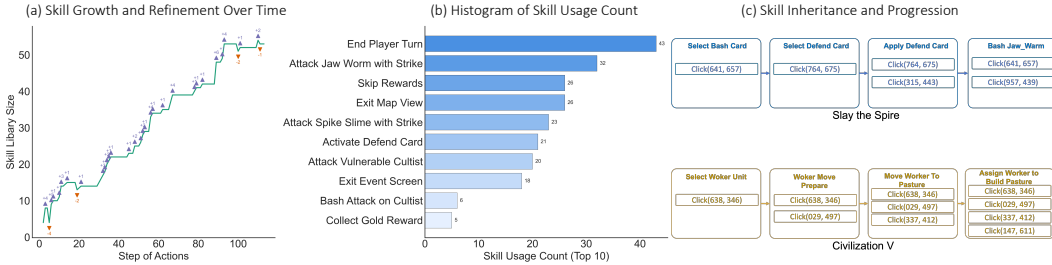


Figure 4: **Analysis of skill evolution and reuse.** (a) Skill library size increases over time through augmentation (+) and pruning (-). (b) Top-10 most frequently invoked skills in *Slay the Spire*. (c) Examples of compositional skill inheritance across environments, showing how higher-level routines are built from atomic actions.

The primary challenge in progressing through both games lies in executing two to three precise actions in sequence. For example, in *Slay the Spire*, one of the most demanding skills involves dragging a card to accurately target and attack a monster. In *Civilization V*, the key difficulty is controlling a unit to move or attack enemy. All baseline agents are capable of interpreting visual observations, but consistently fail to execute correct actions based on that understanding. In particular, the baseline UITARS-1.5 [32] is able to perceive game progression and articulate the next intended action. However, due to overfitting to its training data, the output action coordinates are often incorrect and misaligned with the unseen game interface.

5.3 Ablation Studies

We conduct ablation studies to analyze two core aspects of our framework: the effectiveness of skill evolution over time and the contribution of individual modules to overall performance.

Skill Evolution over Rounds. As shown in Table 2, the agent undergoes four training rounds, each consisting of 100 steps. We limit each round to 100 steps to prevent the character from being defeated

Table 3: **Ablation study of core components in the bottom-up agent.** We assess the contribution of three key modules: visual change filtering during skill augmentation, MCTS-based selection during skill invocation, and LLM-generated skill description during skill refinement. Fully removing the entire module halts progression entirely, so we ablate a component of each module individually. Metrics include gameplay progression, in-game score, execution effectiveness, and token efficiency.

Setting	Visual Change Filter	MCTS Selection	Skill Description	Progression $\uparrow$ (Floors Cleared)	In-game $\uparrow$ Scores	Execution $\uparrow$ Responsive Rate	Token Costs $\downarrow$ (USD \$)
w/o Visual Filter		✓	✓	5	33	89.29%	\$ 2.53
w/o MCTS	✓		✓	1	5	64.52%	\$ 1.48
w/o Description	✓	✓		5	22	92.77%	\$ 2.15
Full Module (Ours)	✓	✓	✓	6	36	93.14%	\$ 2.66

by enemies—longer episodes often result in premature termination. Despite this constraint, we observe clear signs of skill evolution: the skill library grows steadily, with new skills added and non-functional ones pruned. Execution effectiveness and game score improve consistently across rounds, eventually converging in Round 4. This supports our claim that skill reuse and refinement lead to measurable behavioral improvement over time. Figure 4 illustrates the internal dynamics of bottom-up skill evolution. More analysis of Figure 4 can be found in Subsection B.1.

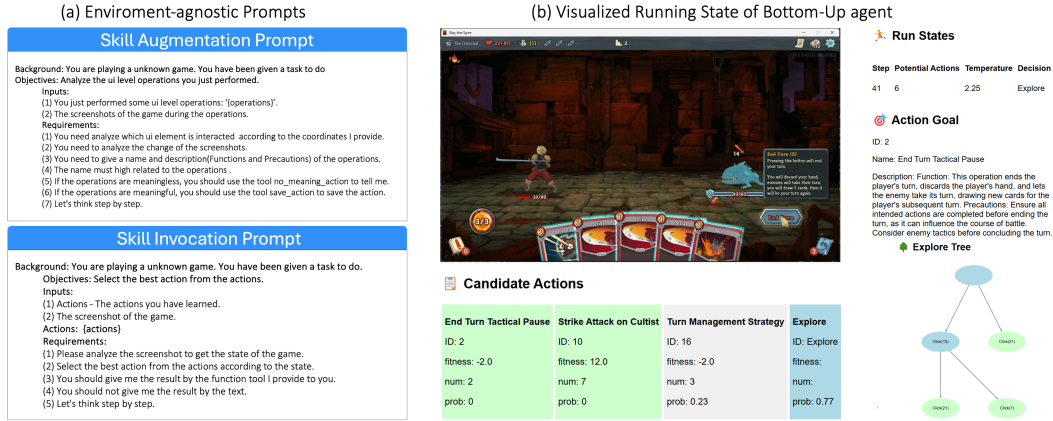


Figure 5: **Prompting and execution visualization of the bottom-up agent.** (a) Environment-agnostic prompts used for skill augmentation and invocation, enabling reasoning without access to game-specific APIs. (b) We design a GUI to visualize execution state of the agent during gameplay, showing candidate actions, selected goal, reasoning metadata and the corresponding skill plan tree.

Effect of Core Modules. To evaluate each module’s contribution, we ablate one component at a time: visual change filtering during augmentation, MCTS-based selection during invocation, and LLM-generated skill description during refinement. As shown in Table 3, removing any single module impairs performance across progression, in-game scores, and execution responsiveness, though to varying degrees. Notably, removing MCTS results in near-complete failure to progress, indicating its critical role in long-horizon skill planning. Disabling visual change filtering inflates token usage and reduces effective execution rate, as many invalid skills are retained. Without semantic description, the agent struggles to consolidate behaviors, leading to lower in-game scores and library reuse.

6 Limitations, Challenges, and Future Work

While this work offers a proof of concept for the bottom-up paradigm, building a practical and scalable agent system remains an open challenge. Our current implementation shows that learning from interaction alone is feasible, but it is not yet a comprehensive solution. Below, we outline key limitations and open problems for future work.

Exploration Overhead. One current limitation of our bottom-up agent is its exploration efficiency. Since skills must be discovered and validated purely through trial-and-reasoning without task-specific priors or APIs, the agent typically requires 2-2.5 times more environment interactions to reach comparable progression levels relative to prior-assisted baselines. For example, baseline agents require approximately 6 hours to complete 1,000 interaction steps in either game, whereas our

bottom-up agents take around 12 hours due to the additional reasoning and skill refinement processes involved. This additional exploration cost is a direct consequence of the zero-prior setting and the need to bootstrap useful behaviors from scratch. Future work could mitigate this overhead through more efficient skill proposal mechanisms, transfer across similar environments, or memory-based generalization from past deployments.

Reset and Evaluation Protocols. A fundamental challenge in open-ended environments is the lack of reliable resetting mechanisms. Unlike benchmark tasks with well-defined episode boundaries and reproducible seeds, open-ended games often lack fine-grained control over initial conditions. This introduces significant variance during evaluation and makes systematic comparisons across agents or iterations difficult. Developing standardized protocols for episode resetting and controlled skill re-evaluation will be critical for more robust benchmarking of bottom-up systems.

Perception of Subtle Environmental Changes. Our current reward modeling relies on perceptible visual changes (e.g., GUI transitions or progression cues) to trigger semantic credit assignment. However, many meaningful behaviors—such as defensive preparations or long-term setups—produce only subtle or delayed visual signals. The current framework may fail to detect or reward such latent strategies. A promising direction is to combine reinforcement learning techniques with implicit reward shaping, where agents learn to attribute credit over extended horizons and from nuanced patterns of environmental feedback.

Implicit Rewards and Strategic Refinement. Despite these limitations, the current framework already enables agents to discover skills that effectively advance the game—e.g., clearing turns, acquiring items, or defeating enemies. However, further refinement requires transitioning from reactive behaviors to long-term, strategic play. Since no explicit reward signals exist, agents must rely on delayed environmental changes to evaluate skill quality. Introducing reinforcement learning techniques may help uncover deeper reward structures and promote strategy-level improvements, enabling skills to evolve not just in form, but in function.

Skill Abstraction and Functionalization. While our agents can accumulate reusable routines via record-and-replay, these skills remain as flat action sequences rather than modular, parameterized functions. Abstracting such routines into callable, general-purpose skills is fundamentally difficult under a zero-prior setting—without predefined APIs or environment structure, it is unclear which parts of a skill should be parameterized, and even harder to infer valid parameter values from raw interaction. Without this abstraction, skill reuse remains limited to surface-level repetition rather than generalizable logic.

Asynchronous Multi-Agent Skill Evolution. Our framework assumes a shared but globally consistent skill library across agents. However, when multiple agents explore asynchronously, conflicts may arise: concurrent edits, incompatible refinements, or redundant additions. Future work should explore decentralized coordination mechanisms—such as eventual consistency protocols, versioned skill records, or trust-weighted refinement consensus—to ensure coherent library evolution in massively parallel deployments.

These limitations underscore that bottom-up agent design is still in its early stages. Addressing these challenges will be essential to scaling the framework from proof-of-concept to widely deployable systems.

7 Conclusion

In this work, we revisit LLM-based agent design by framing existing systems within a top-down paradigm, where agents act as architects to execute human-engineered goals, subtasks, and workflows. While effective in closed settings, such agents struggle to generalize beyond predefined structures. To address this, we propose a bottom-up paradigm, where agents function as explorers: acquiring, refining, and reusing skills through interaction, guided solely by implicit environmental feedback. We formalize this paradigm and implement it as a working system, demonstrating that agents can build competence autonomously from raw experience.

Rather than replacing top-down approaches, our method complements them by enabling skill discovery in environments lacking predefined goals and APIs. In two open-ended games, our bottom-up agents exhibit emergent competence and behavioral efficiency, despite starting with no prior knowledge. Though only a proof of concept, our system points toward a promising direction for agent

development. The bottom-up paradigm envisions a future where millions of agents operate across diverse environments. They share a unified and evolving skill library. The true value of agentic systems lies not in barrier-free workflow design, but in an experience-driven skill library continually honed by the collective experience of large-scale, real-world deployment.

Broader Impacts

This work presents a bottom-up agent framework for autonomous skill acquisition through interaction, without relying on human-defined goals or APIs. While it enables scalable learning in open-ended environments and reduces dependence on manual supervision, uncontrolled deployment may lead to unintended behaviors if implicit rewards misalign with human intent. As the framework scales to real-world domains, safety, oversight, and skill traceability will become critical considerations. This study is confined to virtual environments; no real-world deployment is conducted.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. Do as i can, not as i say: Grounding language in robotic affordances, 2022. URL <https://arxiv.org/abs/2204.01691>.
- [3] Anthropic. Claude 3.7 sonnet and claude code. *Anthropic Blog*, February 2025. <https://www.anthropic.com/news/claude-3-7-sonnet>.
- [4] Bowen Baker, Ilge Akkaya, Peter Zhokov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos. *Advances in Neural Information Processing Systems*, 35:24639–24654, 2022.
- [5] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, et al. Windows agent arena: Evaluating multi-modal os agents at scale. *arXiv preprint arXiv:2409.08264*, 2024.
- [6] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023. URL <https://arxiv.org/abs/2307.15818>.
- [7] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Utsav Malla, Deeksha Manjunath, Igor Mordatch, Ofir Nachum, Carolina Parada, Jodilyn Peralta, Emily Perez, Karl Pertsch, Jornell Quiambao, Kanishka Rao, Michael Ryoo, Grecia Salazar,

- Pannag Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Sontakke, Austin Stone, Clayton Tan, Huang Tran, Vincent Vanhoucke, Steve Vega, Quan Vuong, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-1: Robotics transformer for real-world control at scale, 2023. URL <https://arxiv.org/abs/2212.06817>.
- [8] Butterfly Effect AI. Manus ai: General ai agent, March 2025. URL <https://manus.im/>. Accessed: 2025-05-14.
 - [9] Vivien Cabannes, Charles Arnal, Wassim Bouaziz, Xingyu Yang, Francois Charton, and Julia Kempe. Iteration head: A mechanistic study of chain-of-thought. *Advances in Neural Information Processing Systems*, 37:109101–109122, 2024.
 - [10] Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, et al. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657*, 2025.
 - [11] Xiaokang Chen, Zhiyu Wu, Xingchao Liu, Zizheng Pan, Wen Liu, Zhenda Xie, Xingkai Yu, and Chong Ruan. Janus-pro: Unified multimodal understanding and generation with data and model scaling. *arXiv preprint arXiv:2501.17811*, 2025.
 - [12] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures, 2018. URL <https://arxiv.org/abs/1802.01561>.
 - [13] Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity is all you need: Learning skills without a reward function, 2018. URL <https://arxiv.org/abs/1802.06070>.
 - [14] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-rl: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
 - [15] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
 - [16] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, pages 1–7, 2025.
 - [17] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. *arXiv preprint arXiv:2401.13919*, 2024.
 - [18] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 3(4):6, 2023.
 - [19] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations, 2025. URL <https://arxiv.org/abs/2412.14803>.
 - [20] Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yongqiang Dou, Yanjun Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi Fan. Vima: General robot manipulation with multimodal prompts, 2023. URL <https://arxiv.org/abs/2210.03094>.
 - [21] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4015–4026, 2023.
 - [22] Joel Z. Leibo, Edgar Duéñez-Guzmán, Alexander Sasha Vezhnevets, John P. Agapiou, Peter Sunehag, Raphael Koster, Jayd Matyas, Charles Beattie, Igor Mordatch, and Thore Graepel. Scalable evaluation of multi-agent reinforcement learning with melting pot, 2021. URL <https://arxiv.org/abs/2107.06857>.

- [23] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society, 2023. URL <https://arxiv.org/abs/2303.17760>.
- [24] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control, 2023. URL <https://arxiv.org/abs/2209.07753>.
- [25] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- [26] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [27] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning, 2016. URL <https://arxiv.org/abs/1602.01783>.
- [28] OpenAI. Gpt-4o: Openai’s multimodal flagship model. *OpenAI Blog*, May 2024. <https://openai.com/index/hello-gpt-4o/>.
- [29] OpenAI. Introducing operator: Openai’s autonomous web agent. *OpenAI Blog*, January 2025. <https://openai.com/index/introducing-operator/>.
- [30] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- [31] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2023.
- [32] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.
- [33] Christopher Rawles, Sarah Clinckemaulle, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024.
- [34] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- [35] Omar Shaikh, Hongxin Zhang, William Held, Michael Bernstein, and Diyi Yang. On second thought, let’s not think step by step! bias and toxicity in zero-shot reasoning. *arXiv preprint arXiv:2212.08061*, 2022.
- [36] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.
- [37] David Silver and Richard S Sutton. Welcome to the era of experience. 2025.
- [38] Austin Stone, Ted Xiao, Yao Lu, Keerthana Gopalakrishnan, Kuang-Huei Lee, Quan Vuong, Paul Wohlhart, Sean Kirmani, Brianna Zitkovich, Fei Xia, et al. Open-world object manipulation using pre-trained vision-language models. *arXiv preprint arXiv:2303.00905*, 2023.

- [39] Hongjin Su, Ruoxi Sun, Jinsung Yoon, Pengcheng Yin, Tao Yu, and Serkan Ö. Arık. Learn-by-interact: A data-centric framework for self-adaptive agents in realistic environments, 2025. URL <https://arxiv.org/abs/2501.10893>.
- [40] Maciej Świechowski, Konrad Godlewski, Bartosz Sawicki, and Jacek Mańdziuk. Monte carlo tree search: A review of recent modifications and applications. *Artificial Intelligence Review*, 56(3):2497–2562, 2023.
- [41] Open Ended Learning Team, Adam Stooke, Anuj Mahajan, Catarina Barros, Charlie Deck, Jakob Bauer, Jakub Sygnowski, Maja Trebacz, Max Jaderberg, Michael Mathieu, Nat McAleese, Nathalie Bradley-Schmieg, Nathaniel Wong, Nicolas Porcel, Roberta Raileanu, Steph Hughes-Fitt, Valentin Dalibard, and Wojciech Marian Czarnecki. Open-ended learning leads to generally capable agents, 2021. URL <https://arxiv.org/abs/2107.12808>.
- [42] Alexander Sasha Vezhnevets, Simon Osindero, Tom Schaul, Nicolas Heess, Max Jaderberg, David Silver, and Koray Kavukcuoglu. Feudal networks for hierarchical reinforcement learning, 2017. URL <https://arxiv.org/abs/1703.01161>.
- [43] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [44] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. *arXiv preprint arXiv:2305.04091*, 2023.
- [45] Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. Paired open-ended trailblazer (poet): Endlessly generating increasingly complex and diverse learning environments and their solutions, 2019. URL <https://arxiv.org/abs/1901.01753>.
- [46] Zhexiong Wang, Yutong Wang, Xuebo Liu, Liang Ding, Miao Zhang, Jie Liu, and Min Zhang. Agentdropout: Dynamic agent elimination for token-efficient and high-performance llm-based multi-agent collaboration. *arXiv preprint arXiv:2503.18891*, 2025.
- [47] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.
- [48] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [49] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL <https://arxiv.org/abs/2308.08155>.
- [50] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osvorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.
- [51] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [52] Hui Yang, Sifu Yue, and Yunzhong He. Auto-gpt for online decision making: Benchmarks and additional opinions. *arXiv preprint arXiv:2306.02224*, 2023.
- [53] Mingyu Yang, Yaodong Yang, Zhenbo Lu, Wengang Zhou, and Houqiang Li. Hierarchical multi-agent skill discovery. *Advances in Neural Information Processing Systems*, 36:61759–61776, 2023.

- [54] Shunyu Yao. The second half. *ysymyth.github.io*, April 2025. <https://ysymyth.github.io/The-Second-Half/>.
- [55] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [56] Yuting Zeng, Weizhe Huang, Lei Jiang, Tongxuan Liu, Xitai Jin, Chen Tianying Tiana, Jing Li, and Xiaohua Xu. S²-mad: Breaking the token barrier to enhance multi-agent debate efficiency. *arXiv preprint arXiv:2502.04790*, 2025.
- [57] Lianmin Zheng, Jiacheng Yang, Han Cai, Weinan Zhang, Jun Wang, and Yong Yu. Magent: A many-agent reinforcement learning platform for artificial collective intelligence, 2017. URL <https://arxiv.org/abs/1712.00600>.
- [58] Yifei Zhou, Qianlan Yang, Kaixiang Lin, Min Bai, Xiong Zhou, Yu-Xiong Wang, Sergey Levine, and Erran Li. Proposer-agent-evaluator (pae): Autonomous skill discovery for foundation model internet agents. *arXiv preprint arXiv:2412.13194*, 2024.

Algorithm 1: Bottom-Up Skill Evolution

Input: Environment $(\mathcal{X}, \mathcal{A}, T, \mathcal{R})$, initialized skill library $\mathcal{S} = \emptyset$, LLM model M

```
foreach agent do
  while episode not terminated do
    Observe  $x_t \in \mathcal{X}$ ;
    Sample candidate skills  $\mathbb{S}_t = M(p^{\text{select}}, x_t, \mathcal{S})$ ;
    if  $\mathbb{S}_t = \emptyset$  then
      for  $k = 1$  to  $k_{\max}$  do
        Generate  $s^{(k)} = \text{Augment}(s^{(k-1)}, a_k)$ ;
        if  $\text{effect}(s^{(k)})$  is recognizable then
          Generate  $d_\sigma = M(p^{\text{describe}}, (x_t, s^{(k)}, \mathcal{T}))$ ;
          Update  $\mathcal{S} \leftarrow \mathcal{S} \cup \{(s^{(k)}, d_\sigma)\}$ ;
          break;
    else
      Evaluate  $\mathbb{S}_t$  with MCTS;
      Execute the best skill  $\sigma^*$  from MCTS;
      Observe trajectory  $\mathcal{T}$  and compute  $R_{\text{semantics}}$ ;
      if  $R_{\text{semantics}}$  low across agents then
        Remove  $\sigma$  from  $\mathcal{S}$ ;
      if skill count  $<$  threshold then
        Refine skill:  $\sigma' = M(p^{\text{refine}}, (x_t, \sigma, \mathcal{T}))$ ;
        Replace  $\sigma$  with  $\sigma'$  if improvement observed;
```

A More Related Works

Learning Agents in Traditional Reinforcement Learning. Deep reinforcement learning has produced agents capable of learning complex skills from low-level inputs through trial-and-error based on predefined rewards. Notably, DQN demonstrated that a convolutional agent could achieve human-level play on multiple Atari games, with the game score as the sole reward signal [26]. Asynchronous methods like A3C allowed actor-critic agents to surpass Atari benchmarks [27], and PPO further refined policy-gradient updates for improved reliability [34]. Scalable architectures such as IMPALA extended RL to hundreds of tasks, enabling positive knowledge transfer in a single agent trained on 3D and Atari games [12]. These works highlight the power of trial-and-error learning in well-defined environments. However, they struggle with transferability across environments, as they rely on task-specific reward functions and cannot easily generalize beyond the training scenario.

Classic hierarchical RL frameworks like FeUdal Networks (FuN) introduce skill abstraction by separating a “manager” that sets goals from a “worker” that executes primitive actions, yet still require predefined task hierarchies and tuning [42]. Unsupervised skill discovery methods like DIAYN maximize diversity through self-discovered behaviors without explicit rewards [13]. The bottom-up agent framework extends these ideas by accumulating behaviors into a global knowledge base, where skills are continuously refined and shared across tasks. This method not only addresses skill transferability but also reduces reliance on task-specific rewards, supporting cross-task skill reuse by design.

Open-Ended and Multi-Agent Learning Systems. Open-ended learning frameworks have shown the potential of dynamically generating new tasks and environments for agents, leading to more complex and adaptive behaviors over time. POET generates evolving obstacles and reuses agents across tasks, enabling continual progress in new environments [45]. Similarly, DeepMind’s XLand creates procedurally generated 3D worlds, where agents learn without human intervention and exhibit general behaviors like experimentation and innovation [41]. Multi-agent systems, such as Melting Pot, extend these ideas by simulating complex environments with multiple interacting agents, allowing for emergent behaviors to evolve over time [22]. However, these systems often rely on centralized coordination, role assignment, and predefined communication protocols, limiting their flexibility and adaptability to new, unexpected tasks.

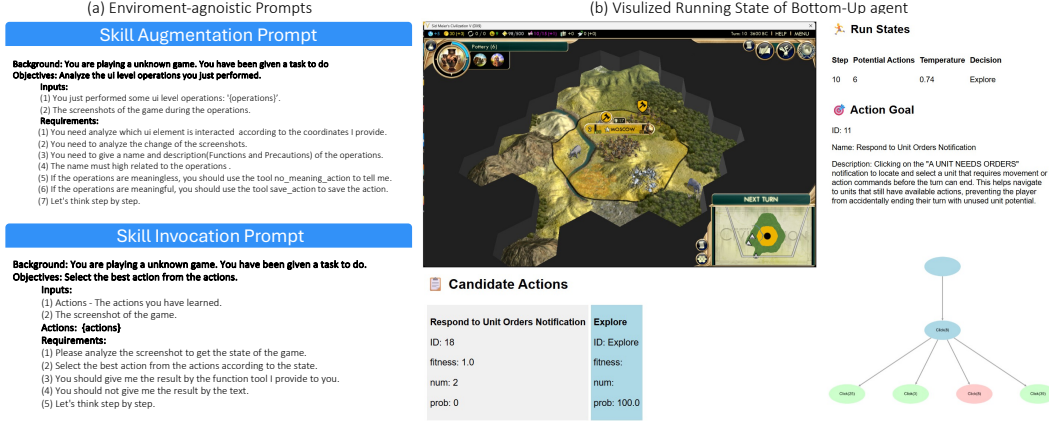


Figure 6: **Prompting and execution visualization of the bottom-up agent.** (a) Environment-agnostic prompts used for skill augmentation and invocation, enabling reasoning without access to game-specific APIs. We use the same codebase and prompts in both Slay the Spire and Civilization V. (b) The GUI to visualize execution state of the agent playing Civilization V.

Many multi-agent RL platforms [53], like MAgent still assume a fixed scenario or symmetric roles for agents[57]. Recent multi-agent coordination frameworks for LLMs take an even more structured approach: frameworks like AutoGen and CAMEL define specialized agent roles and have them converse to solve tasks[49, 23]. In these cases, the multi-agent interaction protocol or hierarchy is designed a priori—whether it’s a communication channel, a population update rule, or an explicit role assignment. Rather than relying on fixed roles and local synchronization, bottom-up agent framework shares learned skills asynchronously in a global repository. This decentralized, open-ended approach allows agents to operate independently, continuously exploring and learning, while benefiting from the discoveries of others without predefined roles, providing greater flexibility and scalability.

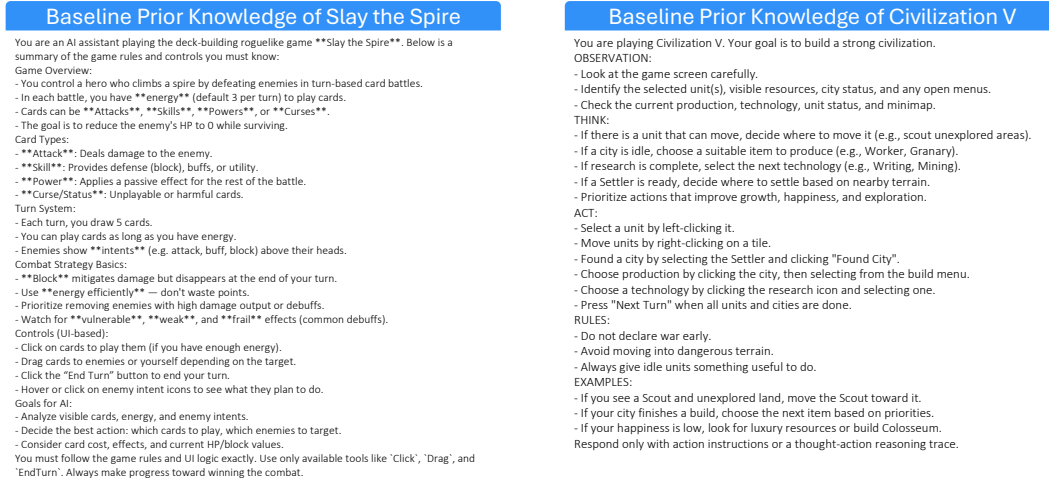


Figure 7: **Baseline prior knowledge provided to top-down agents.** To enable task execution in Slay the Spire (left) and Civilization V (right), baseline agents are given structured game-specific summaries detailing rules, objectives, and UI controls. These priors are necessary for baseline agents to function in the absence of direct API access.

Embodied and Vision-Language-Action Agents. Vision-language-action (VLA) systems, such as RT-1 and RT-2, have achieved remarkable results by combining large-scale pretraining with task-specific learning to perform complex robotic actions based on visual inputs and language commands. RT-1 uses a large dataset of human demonstrations to learn multi-task policies, and RT-2 extends this by incorporating vision-language pretraining from internet-scale data, enabling zero-shot generalization and task completion beyond prior experience [7, 6]. Similarly, PaLM-

SayCan integrates an LLM as a high-level planner that decomposes user instructions into feasible actions, while Code-as-Policies generates executable code for robots to follow complex instructions [2, 24]. More recent approaches, such as VIMA, train transformer-based models on multimodal inputs (images and text) to perform manipulation tasks with minimal supervision, while VPP uses video prediction models to understand physical dynamics for multi-task robotic control [20, 19, 58]. These systems excel at following structured commands but heavily depend on human-engineered demonstrations, predefined skills, and large datasets.

Refine Prompt - Cluster	Refine Prompt - Merge
Background: You are an assistant for grouping similar actions. Input: (1) a JSON array called "new_actions". Instruction: (1) Identify actions that are nearly identical in meaning or function, even if their expressions differ. (2) Group together actions that essentially perform the same task or behavior, even if worded differently. (3) Do not group actions that express different functions or intentions, even if they appear related. (4) Think of this as merging duplicates or near-duplicates, not broad semantic clustering. Output: Exactly one function_call to function "cluster_actions" with no extra text. Here is the list of new_actions (id, name, description): {actions} Please: 1. Identify cluster of **functionally equivalent** actions. 2. For each cluster, select a representative "action_name" and "action_description". 3. List its members as an array of action ids.	Background: You are an assistant that merges and clusters actions in one call. Input: (1) existing_action_clusters: clusters with cluster_id, name, description, members (2) new_actions: raw actions with id, name, description Instruction: (1) Cluster new_actions among themselves by semantic similarity. (2) For each resulting new cluster: - If it matches an existing cluster, merge into it (reuse that cluster_id). - Otherwise, assign cluster_id = -1. (3) Each output cluster must include: - cluster_id - action_name & action_description (representative) - members: combined list of all action IDs. Output: Exactly one function_call to "merge_actions", no extra text. existing_action_clusters: {existing_action_clusters} new_actions: {new_action} Please perform the merge as specified. Return the merged list under the key "clusters".

Figure 8: **Prompts for skill augmentation via clustering and merging.** The left prompt guides the agent to group functionally equivalent actions into clusters; the right prompt extends this by merging with existing clusters. These prompts enable semantic consolidation of skills during augmentation, reducing redundancy and promoting generalization.

The bottom-up framework diverges from these VLA models by focusing on autonomous skill discovery through trial-and-error learning, using visual feedback as the primary source of guidance. While VLA systems leverage pre-existing knowledge or external instructions to guide behavior, our agents develop competencies by interacting with their environment directly, refining behaviors over time. Although our agents incorporate LLM APIs for reasoning and planning, they do not rely on explicit task prompts or large pre-existing datasets for skill acquisition. Instead, they abstract successful behaviors through their own experiences, enabling the emergence of transferable skills without the need for extensive human input.

Learn-by-Interact [39] synthesizes task-aligned data in realistic UI environments using documentation and backward construction. Our work, by contrast, adopts a task-free, bottom-up approach where agents acquire and refine skills purely through interaction. While their targets instruction-following via data synthesis, ours enables open-ended skill evolution without priors—making the two approaches complementary in advancing general-purpose agents.

B More Experimental Results

B.1 More analysis

We analyze the details of Figure 4 here. As shown in (a), the skill library gradually expands through iterative augmentation and pruning, with semantically redundant or ineffective skills removed over time. Panel (b) highlights emergent reuse, where certain high-utility skills are invoked frequently, while the majority of skills are seldom to be invoked. Panel (c) shows how low-level atomic actions are composed into higher-level routines through inheritance, forming modular, environment-specific skills in both Slay the Spire and Civilization V.

B.2 Prompts Demonstration

We demonstrate more prompts we used for baselines and our bottom-up agents. Figure 7 shows the task-specific prompts for baselines, which encode structured prior knowledge about each game. These include explicit rules, action types, UI conventions, and strategic heuristics required for baseline agents to function in the absence of APIs. In contrast, our bottom-up agents cannot access any prior

knowledge to verify the environment’s rules, semantics, or action space. As shown in [Figure 5](#), they rely on general-purpose prompts to analyze, abstract, and organize skills purely from visual interaction. These prompts guide the agent to identify functionally equivalent actions, cluster them into semantically meaningful groups, and iteratively refine the skill library over time—enabling autonomous skill evolution without environment-specific guidance. We use the prompts shown in [Figure 8](#) to cluster and merge skills that are augmented during exploration. As the bottom-up agent interacts with the environment, it accumulates a large number of low-level skills composed of atomic UI actions. Many of these skills are functionally redundant or differ only in superficial details such as coordinates or UI layout. To manage this growing skill library and promote generalization, we periodically invoke LLM-based clustering through structured prompts.